

```
CREATE TABLE accounts (username varchar(100), password
chkpass);
INSERT INTO accounts(username, "password") VALUES ('user1',
'pass1');
INSERT INTO accounts(username, "password") VALUES ('user2',
'pass2');
```

We can authenticate users with a query like the one that follows:

```
SELECT count(*) from accounts where username='user1' and
password = 'pass1'
```

The '=' operator uses the *eq(column_name, text)* in the module to test for equality. *Chkpass* uses the Unix *crypt()* function, and hence it is weak; only the first eight characters of the text are used in the algorithm. *Chkpass* has limited practical use; the *pgcrypto* module is an effective alternative.

Using *fuzzystmatch*

This module installs the *soundx()*, *difference()*, *levenshtein()* and *metaphone()* functions. *Soundx()* and *metaphone()* are phonetic algorithms—they convert a text string to a code string based on its pronunciation. *Difference()* and *levenshtein()* return a numeric value based on the similarity of the two input strings. Let's now look into the *levenshtein()* and *metaphone()* functions. The Levenshtein distance between two strings is the minimum number of insertions, deletions or substitutions required to convert one string to another.

```
SELECT levenshtein('foodlets', 'booklets');
```

This query returns 2, as is obvious.

The *metaphone()* function takes a text string and the maximum length of the output code as its two input parameters. These examples return FTLS:

```
SELECT metaphone('foodlets', 6);
SELECT metaphone('fudlets', 6);
```

If we try to get the Levenshtein distance between the returned strings, this returns 0:

```
SELECT levenshtein('FTLS','FTLS');
```

This means that the two words sound similar.

Fuzzystmatch is very helpful in implementing the search feature for a website. Now the search can work with alternate spellings and misspelled keywords. Reminds you of the 'Did you mean...' feature on Google Search, right?

Using *isbn*

This module will introduce data types to store international standard numbers like International Standard Book Numbers (ISBN), International Standard Music Numbers (ISMN), International Standard Serial Numbers (ISSN), Universal Product Codes (UPC), etc. It will also add functions to validate data, type-cast numbers from older formats to the newer 13-digit formats, and vice-versa. Let's test this module for storing book information:

```
CREATE TABLE books(number isbn13, title varchar(100))
INSERT INTO books("number", title) VALUES ('978-03',
'Rework');
```

The INSERT statement throws an error: *Invalid input syntax for ISBN number: "978-03"*. However, this works just fine:

```
INSERT INTO books("number", title) VALUES ('978-0307463746',
'Rework');
```

To convert a 10-digit ISBN to 13 digits, use the *isbn13()* function:

```
INSERT INTO books("number", title) VALUES
(isbn13('0307463746'), 'Rework');
```

(Actually, the name of the book mentioned here, 'Rework' by Jason Fried, happens to be my favourite book on product/project management! I have prescribed it to all my team-mates.)

Using *hstore*

You must have heard enough about NoSQL and key-value databases. It's not always NoSQL vs relational databases—with the *hstore* module, PostgreSQL allows you to store data in the form of key-value pairs, within a column of a table. Imagine you are processing spreadsheets and you have no idea about the column headers and the data type of the data in the sheets. That's when *hstore* comes to your rescue! Incidentally, *hstore* takes keys and values as text; the value can be NULL, but not the key. Let's create a table with a column of type *hstore* and insert some rows:

```
CREATE TABLE kv_data( id integer, data hstore)
INSERT into kv_data values
(1, hstore('name', 'amit') || hstore('city', 'bangalore')),
(2, hstore('name', 'raghu') || hstore('age', '26')),
(3, hstore('name', 'ran') || hstore('age', '28'));
```

You can create your own keys like 'height', 'favourite_book', etc. The '||' operator is used for